كلية العلوم
FACULTÉ DES SCIENCES

جامعة مولاي إسماعيل
UNIVERSITÉ MOULAY ISMAÏL

Métaheuristiques pour le problème du voyageur de commerce (TSP)

Compte rendu

Master Intelligence Artificielle et Recherche Opérationnelle
Module : Métaheuristiques

Réalisé par :

Amal AABOUR

Encadré par :

Dr. Mustapha OUDANI

Année universitaire : 2025–2026

Compte rendu

Table des matières

I Présentation du problème du voyageur de commerce **2**
1 Définition du problème . . . 2
2 Modélisation par un graphe complet . . . 2
3 Complexité du problème . . . 2
4 Intérêt des heuristiques et métaheuristiques . . . 3

II Choix des instances expérimentales **3**
1 Petites instances . . . 3
1.1 Description des instances . . . 3
1.2 Visualisation des instances . . . 4
2 Instances moyennes . . . 4
2.1 Description des instances . . . 4
2.2 Visualisation des instances . . . 5
3 Grandes instances . . . 6
3.1 Description des instances . . . 6
3.2 Visualisation des instances . . . 6
4 Construction de la matrice des distances . . . 8
5 Justification du choix . . . 8

III Méthodes de résolution du problème **8**
1 Méthode heuristique : plus proche voisin . . . 8
1.1 Construction de la tournée . . . 8
1.2 Influence de la ville de départ . . . 9
1.3 Approche multi-start . . . 9
1.4 Résultats expérimentaux pour les petites instances . . . 9
1.5 Résultats expérimentaux pour les instances moyennes . . . 10
1.6 Résultats expérimentaux pour les grandes instances . . . 11
2 Méthode métaheuristique : recherche tabou . . . 12
2.1 Principe de la recherche tabou . . . 12
2.2 Recherche tabou avec solution initiale aléatoire . . . 13
2.3 Initialisation de la recherche tabou par la solution NN (Warm start) . . . 13
2.4 Intensification séquentielle de la recherche tabou . . . 14
2.5 Recherche tabou hybride avancée . . . 15

IV Analyse des résultats expérimentaux **16**
1 Tableau global des résultats expérimentaux . . . 16
2 Analyse globale des performances . . . 17
3 Analyse par catégorie d'instances . . . 17
4 Impact de la taille des instances sur les performances . . . 18
5 Analyse graphique des performances . . . 18
5.1 Évolution du gap en fonction de la taille de l'instance . . . 19
5.2 Évolution du temps d'exécution selon la taille des instances . . . 20

V Conclusion **21**

VI Annexes **22**

I Présentation du problème du voyageur de commerce

Le problème du voyageur de commerce (*Traveling Salesman Problem*, TSP) est l'un des problèmes d'optimisation combinatoire les plus étudiés en recherche opérationnelle. Il consiste à déterminer la tournée la plus courte permettant de visiter un ensemble de villes exactement une seule fois, avant de revenir à la ville de départ.

1 Définition du problème

Soit un ensemble de n villes et une matrice de distances définissant le coût de déplacement entre chaque paire de villes. Le problème consiste à trouver une permutation des villes minimisant la distance totale parcourue, tout en respectant la contrainte de visite unique de chaque ville.
Mathématiquement, le TSP peut être modélisé par un graphe complet $G = (V, E)$ où :

- V représente l'ensemble des villes ;
- E représente les arêtes reliant chaque paire de villes ;
- chaque arête est associée à un coût correspondant à la distance entre deux villes.

L'objectif est de déterminer un cycle hamiltonien de coût minimal, c'est-à-dire une tournée passant une seule fois par chaque sommet et revenant au point de départ avec une distance totale minimale.

2 Modélisation par un graphe complet

Le problème du voyageur de commerce peut être modélisé à l'aide de la théorie des graphes. Il est représenté par un graphe complet non orienté $G = (V, E)$, où :

- V représente l'ensemble des sommets correspondant aux villes à visiter ;
- E représente l'ensemble des arêtes reliant chaque paire de villes ;
- chaque arête (i, j) est associée à un coût d_{ij} correspondant à la distance entre les villes i et j.

Le graphe est dit complet car chaque ville est reliée à toutes les autres villes. Ainsi, pour un ensemble de n villes, le nombre total d'arêtes est donné par :

$$\frac{n(n-1)}{2}.$$

Dans ce contexte, une solution du problème correspond à un cycle hamiltonien, c'est-à-dire une tournée passant exactement une fois par chaque sommet du graphe avant de revenir au point de départ. L'objectif est de déterminer le cycle hamiltonien de coût minimal, où le coût total correspond à la somme des distances parcourues entre les villes successives de la tournée.
Cette modélisation sous forme de graphe complet permet de formaliser le problème du TSP et de faciliter l'application d'algorithmes heuristiques et métaheuristiques pour sa résolution.

3 Complexité du problème

Le problème du voyageur de commerce est classé parmi les problèmes NP-difficiles. Cela signifie qu'il n'existe pas d'algorithme connu permettant de trouver la solution optimale en temps polynomial pour des instances de grande taille. Le nombre de tournées possibles croît de manière factorielle avec le nombre de villes, ce qui rend la résolution exacte du problème rapidement impraticable lorsque la taille augmente.

4 Intérêt des heuristiques et métaheuristiques

Face à cette complexité, les méthodes exactes deviennent très coûteuses en temps de calcul pour des instances de taille moyenne ou grande. C'est pourquoi les heuristiques et métaheuristiques sont largement utilisées pour obtenir des solutions de bonne qualité dans un temps raisonnable.
Les heuristiques permettent de construire rapidement une solution initiale, tandis que les métaheuristiques offrent des mécanismes d'exploration permettant d'améliorer ces solutions et d'éviter les minima locaux. Dans ce projet, nous utilisons une heuristique gloutonne pour générer une solution initiale, puis une métaheuristique de recherche tabou afin d'améliorer la qualité des tournées obtenues.

II Choix des instances expérimentales

Afin d'évaluer les performances des méthodes proposées pour la résolution du problème du voyageur de commerce (TSP), nous avons utilisé un ensemble d'instances issues de la bibliothèque de référence **TSPLIB**. Cette bibliothèque est largement utilisée dans la littérature scientifique pour tester et comparer les algorithmes d'optimisation combinatoire.
Le choix des instances a été réalisé de manière à couvrir différentes tailles et niveaux de complexité du problème, permettant ainsi une analyse expérimentale représentative.

1 Petites instances

1.1 Description des instances

Les petites instances utilisées dans cette étude proviennent également de la bibliothèque *TSPLIB*. Elles contiennent un nombre relativement réduit de villes et sont souvent utilisées pour vérifier le bon fonctionnement d'un algorithme et analyser son comportement sur des problèmes de taille modérée avant de l'appliquer à des instances plus grandes.

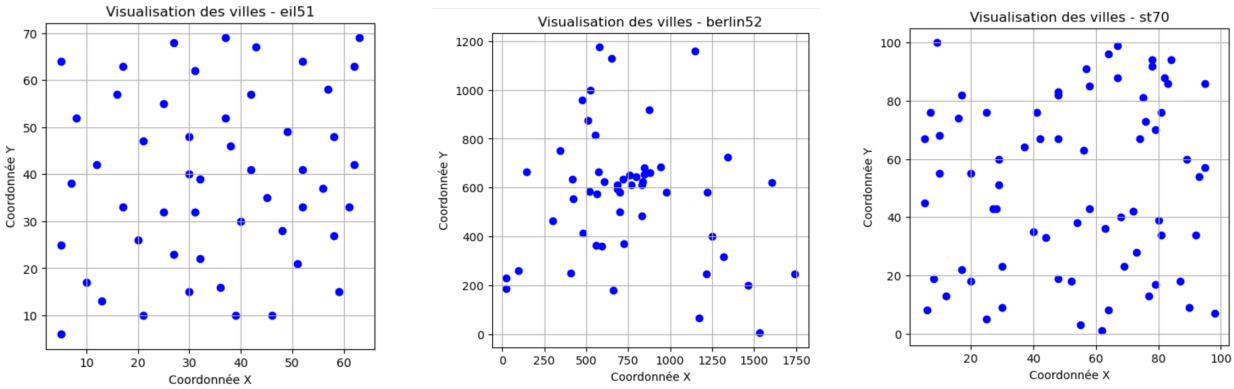
- **eil51** : cette instance contient 51 villes et provient d'un problème de distribution. Les villes sont relativement bien réparties dans le plan, ce qui permet d'obtenir une structure de distances variée tout en conservant une taille de problème raisonnable.
- **berlin52** : cette instance comprend 52 villes représentant des localisations dans la ville de Berlin. Les villes présentent une dispersion assez régulière dans le plan, ce qui en fait une instance très utilisée dans les travaux de recherche sur le problème du voyageur de commerce.
- **st70** : cette instance contient 70 villes. La distribution des villes est légèrement plus étendue que dans les deux instances précédentes, avec une dispersion modérée permettant de tester la capacité de l'algorithme à explorer différentes configurations de tournées.

Table 1: Caractéristiques des petites instances utilisées

Instance	Nombre de villes	Type de distance	Valeur optimale	Dispersion des villes
eil51	51	EUC_2D	426	Villes relativement bien réparties dans le plan avec une distribution équilibrée.
berlin52	52	EUC_2D	7542	Villes représentant des localisations dans la ville de Berlin avec une dispersion régulière.
st70	70	EUC_2D	675	Distribution légèrement plus étendue permettant de tester différentes configurations de tournées.

1.2 Visualisation des instances

Afin de mieux comprendre la structure géographique des instances utilisées, nous présentons une visualisation de la position des villes pour les instances considérées. Cette représentation permet d'observer la distribution spatiale des villes et d'identifier certaines caractéristiques pouvant influencer la difficulté du problème.



(a) eil51 (b) berlin52 (c) st70

Figure 1: Visualisation de la distribution des villes pour les petites instances TSPLIB

La figure 1 illustre la distribution géographique des villes pour les petites instances utilisées dans cette étude. On observe que les villes sont relativement bien réparties dans le plan pour ces instances. Cette dispersion équilibrée permet de tester efficacement le comportement de l'algorithme sur des configurations variées tout en conservant une complexité modérée du problème.

2 Instances moyennes

2.1 Description des instances

Les instances moyennes utilisées dans cette étude proviennent également de la bibliothèque *TSPLIB*. Elles contiennent un nombre plus important de villes que les petites instances et permettent d'évaluer le comportement de l'algorithme sur des problèmes de taille intermédiaire. Ces instances représentent un compromis entre complexité computationnelle et qualité des solutions obtenues.

- **kroA100** : cette instance contient 100 villes issues d'un ensemble de problèmes artificiels fréquemment utilisés pour l'évaluation des algorithmes de résolution du TSP. Les villes sont relativement bien

réparties dans le plan, ce qui génère une dispersion équilibrée et une structure de distances variée.

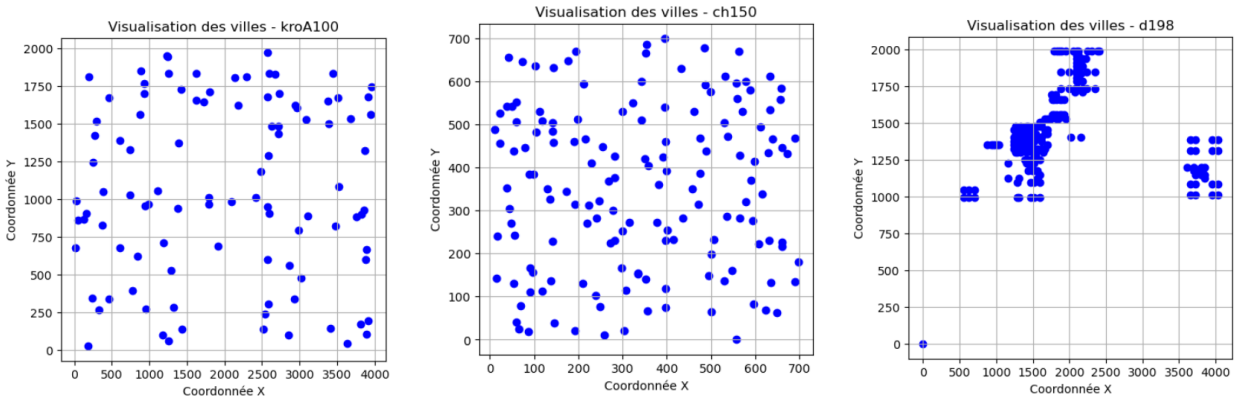
- **ch150** : cette instance comprend 150 villes provenant d'un problème réel lié à la logistique. Les villes présentent une dispersion modérée dans le plan, ce qui permet d'évaluer la capacité de l'algorithme à gérer des configurations plus complexes que celles des petites instances.
- **d198** : cette instance contient 198 villes et constitue un problème de taille intermédiaire plus exigeant. Les villes sont réparties sur une zone relativement étendue avec des distances variées entre les points, ce qui rend l'espace de recherche plus large et augmente la difficulté du problème.

Table 2: Caractéristiques des instances moyennes utilisées

Instance	Nombre de villes	Type de distance	Valeur optimale	Dispersion des villes
kroA100	100	EUC_2D	21282	Villes relativement bien réparties dans le plan avec une dispersion équilibrée.
ch150	150	EUC_2D	6528	Distribution modérée permettant d'évaluer le comportement de l'algorithme sur un problème plus complexe.
d198	198	EUC_2D	15780	Villes réparties sur une zone plus étendue avec des distances variées entre les points.

2.2 Visualisation des instances

Afin d'analyser la structure spatiale des instances de taille moyenne, nous représentons graphiquement la position des villes dans le plan. Cette visualisation permet d'examiner la dispersion des villes et d'identifier certaines caractéristiques géométriques susceptibles d'influencer la difficulté du problème.



(a) kroA100 (b) ch150 (c) d198

Figure 2: Visualisation de la distribution des villes pour les instances moyennes TSPLIB

La figure 2 illustre la distribution géographique des villes pour les instances moyennes utilisées dans cette étude.
On observe que les villes sont réparties sur une zone plus étendue que dans les petites instances. Cette augmentation du nombre de villes entraîne une structure spatiale plus complexe, ce qui peut influencer la difficulté du problème.

Dans les instances kroA100 et ch150, les villes présentent une dispersion relativement homogène dans le plan. En revanche, l'instance d198 montre une concentration plus marquée de certaines villes dans des zones spécifiques, ce qui peut créer des sous-structures dans la tournée optimale et rendre la recherche de solutions plus complexe.

3 Grandes instances

3.1 Description des instances

Les grandes instances utilisées dans cette étude proviennent de la bibliothèque *TSPLIB*, qui constitue une référence standard pour l'évaluation des algorithmes de résolution du problème du voyageur de commerce. Ces instances contiennent plusieurs centaines de villes et permettent d'évaluer la performance des métaheuristiques sur des problèmes de grande taille.

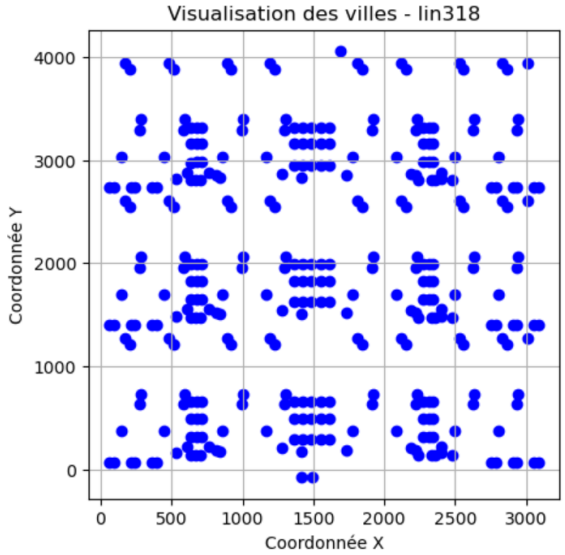
- **rat575** : cette instance contient 575 villes générées aléatoirement dans le plan. Les villes sont relativement uniformément dispersées, ce qui produit une distribution spatiale assez régulière. Ce type d'instance est souvent utilisé pour tester la capacité d'un algorithme à explorer efficacement un espace de recherche étendu.
- **lin318** : cette instance comprend 318 villes et provient d'un problème réel lié à la planification logistique. Les villes présentent une dispersion irrégulière, avec certaines zones plus denses que d'autres, ce qui peut rendre la recherche des meilleures tournées plus complexe.
- **pr439** : cette instance comporte 439 villes issues d'un problème réel de distribution. Les villes sont largement dispersées dans le plan, avec des distances relativement variées entre les points. Cette dispersion peut augmenter la difficulté de l'exploration de l'espace des solutions.
- **pcb442** : cette instance contient 442 villes et provient d'un problème de conception de circuits imprimés (*Printed Circuit Board*). Les villes sont généralement plus concentrées et moins dispersées, car elles représentent des composants situés sur une carte électronique.

Table 3: Caractéristiques des grandes instances utilisées

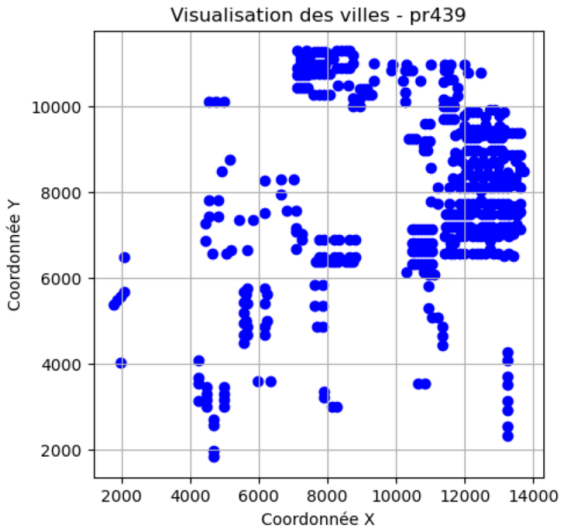
Instance	Nombre de villes	Type de distance	Valeur optimale	Dispersion des villes
rat575	575	EUC_2D	6773	Villes générées aléatoirement et relativement uniformément dispersées dans le plan.
lin318	318	EUC_2D	42029	Dispersion irrégulière avec des zones plus denses que d'autres.
pr439	439	EUC_2D	107217	Villes largement dispersées avec des distances variées entre les points.
pcb442	442	EUC_2D	50778	Villes plus concentrées représentant des composants sur un circuit imprimé.

3.2 Visualisation des instances

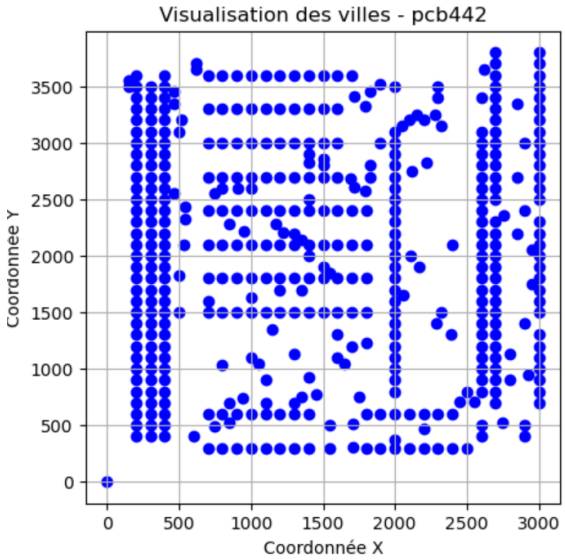
Pour les instances de grande taille, la visualisation des positions des villes permet d'observer des structures spatiales plus complexes. Ces représentations facilitent l'identification de zones de forte concentration de villes ainsi que de régions plus dispersées, ce qui peut influencer la difficulté de recherche des tournées optimales.



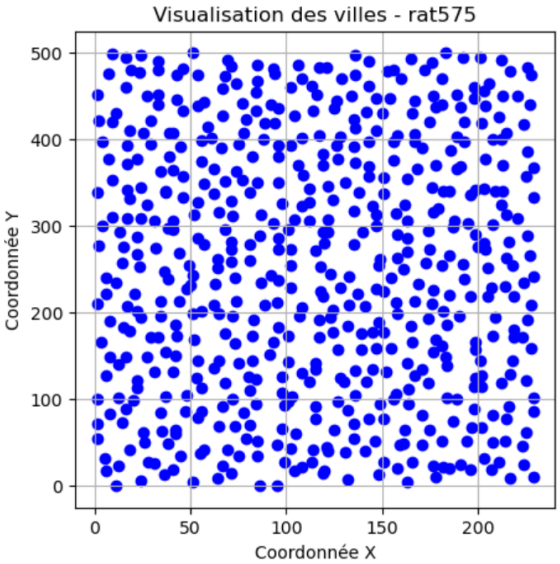
(a) lin318



(b) pr439



(c) pcb442



(d) rat575

Figure 3: Visualisation de la distribution des villes pour les grandes instances TSPLIB

La figure 3 présente la distribution spatiale des villes pour les grandes instances étudiées.
On observe que ces instances présentent des structures géographiques plus complexes que celles des instances de petite et moyenne taille. Certaines instances, comme lin318 et pr439, montrent des regroupements de villes dans certaines régions du plan, ce qui peut influencer la structure des tournées optimales.
L'instance pcb442 présente une organisation particulière liée à son origine dans la conception de circuits imprimés, ce qui explique la disposition plus structurée des villes.
Enfin, l'instance rat575 se caractérise par une distribution plus uniforme et aléatoire des villes dans le plan, ce qui correspond à une génération aléatoire des coordonnées.

4 Construction de la matrice des distances

Afin de résoudre le problème du voyageur de commerce, il est nécessaire de disposer d'une matrice des distances entre toutes les paires de villes.
À partir des coordonnées géographiques fournies dans les fichiers des instances TSPLIB, nous avons construit une matrice de distances pour chaque instance étudiée. Chaque élément de cette matrice représente la distance euclidienne entre deux villes.
La distance entre deux villes i et j, de coordonnées (x_i, y_i) et (x_j, y_j), est calculée à l'aide de la formule suivante :

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

La matrice obtenue est symétrique et sa diagonale principale est composée de zéros, puisque la distance d'une ville à elle-même est nulle. Cette matrice constitue l'entrée principale utilisée par les algorithmes de recherche pour évaluer le coût des tournées.

5 Justification du choix

Le choix de ces instances repose sur plusieurs critères :

- Couvrir un large éventail de tailles afin d'étudier l'impact de la dimension du problème sur les performances ;
- Analyser le comportement des méthodes proposées sur différentes configurations spatiales du problème TSP.

En effet, même pour des instances de tailles proches, la distribution géographique des villes et la nature des distances peuvent influencer significativement la difficulté du problème et la qualité des solutions obtenues.

Après la sélection des instances de test, la section suivante présente les méthodes heuristiques et métaheuristiques implémentées pour résoudre le problème du voyageur de commerce.

III Méthodes de résolution du problème

1 Méthode heuristique : plus proche voisin

1.1 Construction de la tournée

Dans cette première étape, nous avons implémenté l'heuristique du plus proche voisin afin de construire une tournée initiale pour chaque instance étudiée. À partir d'une ville de départ donnée, l'algorithme sélectionne à chaque étape la ville non visitée la plus proche de la ville courante. Cette ville est ensuite ajoutée à la tournée et marquée comme visitée. Le processus est répété jusqu'à ce que toutes les villes soient parcourues. La tournée est finalement fermée en revenant à la ville de départ.
Les distances entre les villes sont calculées à partir de la matrice de distances construite à partir des coordonnées géographiques des villes. Cette approche permet d'obtenir rapidement une solution réalisable pour le problème du voyageur de commerce.
L'heuristique a été appliquée aux différentes instances afin de générer une première solution pour chacune d'elles. Les informations enregistrées pour chaque exécution incluent la tournée obtenue, le coût total de la tournée ainsi que le temps d'exécution de l'algorithme.

1.2 Influence de la ville de départ

La solution obtenue par l'heuristique du plus proche voisin dépend fortement du choix de la ville de départ. En effet, différentes villes de départ peuvent conduire à des tournées de qualité différente.
Afin d'étudier cette influence, nous avons exécuté l'algorithme en utilisant successivement toutes les villes de l'instance comme point de départ. Pour chaque départ possible, une tournée complète est construite et son coût total est calculé. La meilleure solution obtenue parmi tous les départs est ensuite retenue pour l'instance considérée.
Cette expérimentation permet d'évaluer la sensibilité de l'heuristique au choix de la ville initiale et d'identifier les départs conduisant aux solutions les plus efficaces.

1.3 Approche multi-start

Afin d'améliorer la qualité des solutions obtenues, nous avons également appliqué une approche multi-start. Dans cette approche, la ville de départ est choisie de manière aléatoire et l'algorithme est exécuté plusieurs fois.
Pour chaque instance, plusieurs exécutions indépendantes de l'heuristique ont été réalisées avec des villes de départ différentes. Les coûts des tournées obtenues lors de ces exécutions ont ensuite été analysés afin d'évaluer la variabilité des solutions.
Pour chaque instance, nous avons calculé plusieurs indicateurs statistiques, notamment le meilleur coût obtenu, le coût moyen, le pire coût ainsi que l'écart type. Ces indicateurs permettent d'évaluer la robustesse de l'heuristique face au choix de la ville de départ.

1.4 Résultats expérimentaux pour les petites instances

L'heuristique du plus proche voisin a été appliquée aux petites instances sélectionnées dans cette étude, à savoir *eil51*, *berlin52* et *st70*. Les résultats obtenus montrent que la qualité des solutions dépend fortement de la ville de départ choisie.
Dans le cadre de l'approche multi-start, plusieurs exécutions de l'algorithme ont été réalisées afin d'identifier la meilleure solution obtenue pour chaque instance. Les résultats obtenus sont résumés dans le tableau suivant.

Table 4: Résultats de l'heuristique du plus proche voisin pour les petites instances

Instance	Meilleur coût NN	Coût moyen	Pire coût
eil51	505.77	566.39	611.57
berlin52	8182.19	9351.24	10299.37
st70	761.69	826.81	901.27

Les résultats obtenus montrent que l'heuristique du plus proche voisin permet de générer rapidement des solutions réalisables pour les petites instances du problème. Toutefois, la qualité des solutions dépend fortement de la ville de départ choisie.
L'approche multi-start permet d'améliorer les résultats en explorant plusieurs solutions initiales. Pour chaque instance, la meilleure solution obtenue est retenue parmi l'ensemble des exécutions réalisées.
On observe également une certaine variabilité entre les différentes exécutions, comme l'indique l'écart entre le meilleur et le pire coût obtenu. Cette variabilité confirme l'importance du choix de la ville de départ dans l'heuristique du plus proche voisin.

Comparaison avec les solutions optimales pour les petites instances

Afin d'évaluer la qualité des solutions obtenues par l'heuristique du plus proche voisin, les meilleurs coûts obtenus pour les petites instances ont été comparés aux valeurs optimales connues dans la bibliothèque TSPLIB.
Le gap relatif est calculé à l'aide de la formule suivante :

$$gap = \frac{C_{NN} - C_{opt}}{C_{opt}} \times 100$$

où C_{NN} représente le coût de la tournée obtenue par l'heuristique et C_{opt} la valeur optimale connue.

Table 5: Comparaison des résultats NN avec les valeurs optimales pour les petites instances

Instance	Coût NN	Optimal	Gap (%)
eil51	505.77	426	18.73
berlin52	8182.19	7542	8.49
st70	761.69	675	12.84

Les résultats montrent que l'heuristique du plus proche voisin produit des solutions relativement proches des valeurs optimales pour les petites instances. L'instance *berlin52* présente le plus faible écart avec un gap de 8.49%, tandis que l'instance *eil51* présente un écart plus important atteignant 18.73%.
Ces résultats confirment que l'heuristique permet d'obtenir rapidement des solutions réalisables, mais que la qualité des solutions reste limitée par la nature gloutonne de la méthode. Cette observation justifie l'utilisation de méthodes plus avancées, telles que les métaheuristiques, afin d'améliorer les solutions obtenues.

1.5 Résultats expérimentaux pour les instances moyennes

L'heuristique du plus proche voisin a également été appliquée aux instances de taille moyenne sélectionnées dans cette étude, à savoir *kroA100*, *ch150* et *d198*.
Dans un premier temps, l'algorithme a été exécuté pour générer une tournée complète à partir d'une ville de départ donnée. Les résultats obtenus montrent que la qualité des solutions dépend fortement du choix de la ville initiale.
Afin d'améliorer les résultats obtenus, une approche multi-start a été appliquée. Cette approche consiste à exécuter l'heuristique plusieurs fois avec différentes villes de départ et à retenir la meilleure solution obtenue.

Table 6: Résultats NN multi-start pour les instances moyennes

Instance	Meilleur coût	Coût moyen	Pire coût
kroA100	24698.50	27045.21	28694.68
ch150	7078.44	7697.12	8451.16
d198	17809.73	18856.39	20871.82

Afin d'évaluer la robustesse de l'heuristique, plusieurs exécutions ont été réalisées avec des villes de départ choisies aléatoirement. Les résultats obtenus permettent de calculer différents indicateurs statistiques, notamment le meilleur coût, le coût moyen, le pire coût ainsi que l'écart type.

Table 7: Analyse de robustesse de l'heuristique NN pour les instances moyennes

Instance	Meilleur coût	Coût moyen	Pire coût	Écart-type
kroA100	25421.83	27089.43	28694.68	817.28
ch150	7092.71	7761.01	8385.88	311.52
d198	17917.57	18973.75	20871.82	679.36

Afin d'évaluer la qualité des solutions obtenues, les meilleurs coûts ont été comparés aux valeurs optimales connues des instances TSPLIB. Le gap relatif est calculé selon la formule suivante :

$$gap = \frac{C_{NN} - C_{opt}}{C_{opt}} \times 100$$

où C_{NN} représente le coût obtenu par l'heuristique et C_{opt} la valeur optimale connue.

Table 8: Comparaison des résultats NN avec les valeurs optimales pour les instances moyennes

Instance	Coût NN	Optimal	Gap (%)
kroA100	25421.83	21282	19.45
ch150	7092.71	6528	8.65
d198	17917.57	15780	13.55

Les résultats obtenus montrent que l'heuristique du plus proche voisin permet de produire rapidement des solutions réalisables pour les instances de taille moyenne. Toutefois, les écarts par rapport aux solutions optimales restent plus importants que pour les petites instances.
L'instance *ch150* présente le plus faible écart avec un gap de 8.65%, tandis que l'instance *kroA100* présente l'écart le plus élevé atteignant 19.45%. Ces résultats montrent que la qualité des solutions obtenues par l'heuristique diminue progressivement avec l'augmentation de la taille de l'instance.

1.6 Résultats expérimentaux pour les grandes instances

L'heuristique du plus proche voisin a également été appliquée aux instances de grande taille sélectionnées dans cette étude, à savoir *lin318*, *pr439*, *pcb442* et *rat575*.
Comme pour les instances précédentes, une première tournée a été générée à partir d'une ville de départ donnée. Les résultats obtenus montrent que la qualité des solutions produites par l'heuristique dépend fortement du choix de la ville initiale.
Afin d'améliorer les performances de la méthode, une approche multi-start a été utilisée. Cette approche consiste à exécuter l'heuristique plusieurs fois avec différentes villes de départ et à retenir la meilleure solution obtenue pour chaque instance.

Table 9: Résultats NN multi-start pour les grandes instances

Instance	Meilleur coût	Coût moyen	Pire coût
lin318	49215.61	55290.76	58576.89
pr439	127228.36	135096.56	146207.29
pcb442	58952.09	62256.05	66163.73
rat575	7973.24	8297.69	8690.26

Afin d'évaluer la robustesse de l'heuristique, plusieurs exécutions ont été réalisées avec des villes de départ choisies aléatoirement. Les résultats obtenus permettent de calculer différents indicateurs statistiques tels que le meilleur coût, le coût moyen, le pire coût ainsi que l'écart type.

Table 10: Analyse de robustesse de l'heuristique NN pour les grandes instances

Instance	Meilleur coût	Coût moyen	Pire coût	Écart-type
lin318	50458.84	52959.13	54856.45	1022.91
pr439	127283.36	134717.83	144717.18	2966.57
pcb442	68416.27	62565.23	65844.99	1140.12
rat575	8899.63	8289.74	8660.55	133.94

Afin d'évaluer la qualité des solutions obtenues, les meilleurs coûts ont été comparés aux valeurs optimales connues pour les instances TSPLIB. Le gap relatif est calculé selon la formule suivante :

$$gap = \frac{C_{NN} - C_{opt}}{C_{opt}} \times 100$$

où C_{NN} représente le coût obtenu par l'heuristique et C_{opt} la valeur optimale connue.

Table 11: Comparaison des résultats NN avec les valeurs optimales pour les grandes instances

Instance	Coût NN	Optimal	Gap (%)
lin318	50458.84	42029	20.05
pr439	127283.36	107217	18.71
pcb442	68416.27	50778	18.98
rat575	8899.63	6773	19.44

Les résultats obtenus montrent que la qualité des solutions produites par l'heuristique du plus proche voisin diminue lorsque la taille des instances augmente. Les écarts par rapport aux valeurs optimales deviennent plus importants que pour les petites et moyennes instances.
Ces observations s'expliquent par la nature gloutonne de l'heuristique, qui ne permet pas d'explorer efficacement l'espace de recherche lorsque le nombre de villes devient très élevé. Cette limitation justifie l'utilisation de méthodes plus avancées, telles que les métaheuristiques, afin d'améliorer les solutions obtenues.

2 Méthode métaheuristique : recherche tabou

2.1 Principe de la recherche tabou

La recherche tabou est une métaheuristique d'optimisation permettant d'améliorer progressivement une solution initiale en explorant son voisinage. À chaque itération, un ensemble de solutions voisines est généré à partir de la solution courante. La meilleure solution admissible est sélectionnée afin de mettre à jour la solution courante.
Afin d'éviter les cycles et la revisite de solutions récemment explorées, une mémoire appelée *liste tabou* est utilisée. Certains mouvements sont temporairement interdits pendant un nombre limité d'itérations appelé *tenure*. Un critère d'aspiration permet toutefois d'accepter un mouvement tabou si celui-ci conduit à une solution meilleure que la meilleure solution globale trouvée jusqu'à présent.

2.2 Recherche tabou avec solution initiale aléatoire

Dans une première configuration, la recherche tabou est initialisée à partir d'une tournée générée aléatoirement. Cette stratégie correspond à un démarrage à froid (*cold start*) et permet d'évaluer la capacité de la métaheuristique à améliorer une solution initiale de faible qualité.
À chaque itération, un ensemble de mouvements 2-opt est généré afin de produire des solutions voisines. Le meilleur voisin non tabou est sélectionné pour mettre à jour la solution courante. La liste tabou est utilisée pour éviter les cycles et favoriser l'exploration de nouvelles solutions.
Cette première expérience permet d'observer l'amélioration apportée par la recherche tabou par rapport à une solution initiale aléatoire.

Résultats expérimentaux

Table 12: Résultats de la recherche tabou avec solution initiale aléatoire

Instance	Coût obtenu	Temps (s)
eil51	436.39	0.89
berlin52	7958.08	0.66
st70	702.08	0.99
kroA100	21532.86	2.21
ch150	43422.27	4.34
d198	18094.73	3.42
lin318	40611.11	6.67
pr439	12486.48	7.89
pcb442	50921.45	8.20
rat575	8172.28	9.10

Petites instances Pour les petites instances (eil51, berlin52 et st70), la recherche tabou permet d'améliorer significativement la solution initiale générée aléatoirement. L'algorithme converge rapidement vers des solutions de meilleure qualité grâce à l'exploration du voisinage basée sur les mouvements 2-opt. En raison de la taille réduite du problème, l'espace de recherche reste relativement limité, ce qui permet à la recherche tabou d'atteindre rapidement des solutions proches de l'optimum.

Instances moyennes Pour les instances de taille moyenne (kroA100, ch150 et d198), la recherche tabou améliore également la solution initiale aléatoire. Toutefois, l'augmentation du nombre de villes entraîne une croissance importante de l'espace de recherche, ce qui rend le processus d'amélioration plus complexe. Malgré cela, la métaheuristique parvient à réduire significativement le coût de la tournée par rapport à la solution initiale.

Grandes instances Pour les grandes instances (lin318, pr439, pcb442 et rat575), la recherche tabou reste capable d'améliorer les solutions initiales aléatoires, mais l'écart avec l'optimum devient plus important en raison de la complexité du problème. L'espace de recherche étant beaucoup plus vaste, l'algorithme nécessite davantage d'itérations pour explorer efficacement les solutions voisines.

2.3 Initialisation de la recherche tabou par la solution NN (Warm start)

Dans cette configuration, la recherche tabou est initialisée à partir de la meilleure solution obtenue par l'heuristique du plus proche voisin. Cette stratégie correspond à un démarrage à chaud (*warm start*) et permet de commencer la recherche à partir d'une solution déjà relativement proche de l'optimum.

Contrairement à l'initialisation aléatoire, cette approche exploite la solution fournie par l'heuristique NN afin de guider la recherche vers des régions prometteuses de l'espace de recherche. La recherche tabou est ensuite appliquée afin d'explorer le voisinage de cette solution initiale et d'améliorer progressivement la tournée.

Résultats expérimentaux

Table 13: Résultats de la recherche tabou avec initialisation par la solution NN

Instance	Coût Tabou (Warm start)	Gap (%)
eil51	428.98	0.70
berlin52	7544.37	0.03
st70	687.14	1.80
kroA100	21969.74	3.23
ch150	6733.59	3.15
d198	16087.63	1.95
lin318	45318.08	7.83
pr439	116522.78	8.68
pcb442	54935.55	8.19
rat575	7435.89	9.79

L'utilisation de la solution obtenue par l'heuristique du plus proche voisin comme point de départ améliore significativement les performances de la recherche tabou par rapport à l'initialisation aléatoire. En effet, le *warm start* permet de démarrer la recherche à partir d'une solution déjà proche de l'optimum, ce qui réduit le temps nécessaire pour atteindre des solutions de bonne qualité.
Dans le cas du *cold start*, la recherche tabou doit d'abord corriger une solution initiale souvent très éloignée de l'optimum. À l'inverse, l'initialisation par la solution NN permet à la métaheuristique de concentrer la recherche sur l'amélioration locale de la tournée.
Les résultats obtenus montrent que cette stratégie permet d'obtenir des solutions de meilleure qualité et d'accélérer la convergence de l'algorithme, en particulier pour les instances de taille moyenne et grande.

2.4 Intensification séquentielle de la recherche tabou

Dans cette étape, nous introduisons une stratégie d'intensification séquentielle de la recherche tabou. Contrairement à la configuration précédente où la métaheuristique est exécutée une seule fois à partir de la solution obtenue par l'heuristique du plus proche voisin, la solution améliorée produite par la recherche tabou est réutilisée comme solution initiale pour une nouvelle exécution du même algorithme.
Cette approche permet d'exploiter progressivement les améliorations obtenues lors des itérations précédentes et d'explorer plus en profondeur le voisinage des meilleures solutions trouvées. Ainsi, la recherche est guidée vers des régions prometteuses de l'espace de recherche, ce qui peut conduire à des solutions de meilleure qualité.

Résultats expérimentaux

Table 14: Résultats de la recherche tabou après hybridation séquentielle

Catégorie	Instance	Coût Tabou	Valeur optimale	Gap (%)
Small	eil51	428.98	426	0.70
Small	berlin52	7544.37	7542	0.03
Small	st70	687.14	675	1.80
Medium	kroA100	21494.05	21282	1.00
Medium	ch150	6724.03	6528	3.00
Medium	d198	16087.02	15780	1.95
Large	lin318	45021.88	42029	7.12
Large	pr439	116299.12	107217	8.47
Large	pcb442	54492.28	50778	7.31
Large	rat575	7352.09	6773	8.55

Le tableau précédent présente les résultats obtenus après l'application de l'hybridation séquentielle de la recherche tabou.
Pour les petites instances, aucune amélioration supplémentaire n'a été observée par rapport à la configuration précédente. Les solutions obtenues restent identiques, ce qui indique que l'algorithme avait déjà atteint une solution très proche de l'optimum.
Pour les instances de taille moyenne, l'hybridation séquentielle permet d'améliorer légèrement certaines solutions, en particulier pour l'instance *kroA100*, où le gap est réduit à environ 1%. La réutilisation de la solution améliorée comme point de départ permet ainsi d'affiner davantage la recherche.
Enfin, pour les grandes instances, l'hybridation séquentielle contribue également à améliorer les résultats obtenus. Bien que l'écart avec les solutions optimales reste plus important en raison de la taille de l'espace de recherche, l'algorithme parvient à réduire légèrement le gap par rapport aux configurations précédentes.

2.5 Recherche tabou hybride avancée

Afin d'améliorer davantage les performances de la recherche tabou, nous avons développé une version hybride avancée de l'algorithme. Cette approche combine plusieurs mécanismes d'intensification et de diversification afin d'explorer plus efficacement l'espace de recherche. L'algorithme repose sur l'utilisation de mouvements 2-opt pour générer les solutions voisines, tout en intégrant différentes stratégies permettant de guider la recherche vers des modifications plus prometteuses.
Les principales améliorations introduites dans cette version hybride sont les suivantes :

- **Liste de voisins proches** : pour chaque ville, une liste de voisins proches est construite afin de limiter l'exploration du voisinage aux villes les plus pertinentes.
- **Sélection des arêtes longues** : les mouvements 2-opt sont générés en ciblant en priorité les arêtes les plus longues de la tournée, ce qui augmente les chances de trouver des améliorations significatives.
- **Mécanisme de diversification** : lorsqu'aucune amélioration n'est observée pendant plusieurs itérations, un mouvement de type *double-bridge* est appliqué afin de perturber la solution et d'explorer une nouvelle région de l'espace de recherche.
- **Stratégie de redémarrage** : la recherche peut être relancée à partir de la meilleure solution globale trouvée afin d'éviter la stagnation de l'algorithme.

Résultats expérimentaux

Le tableau 15 présente l'ensemble des résultats obtenus.

Table 15: Résultats globaux obtenus par l'algorithme proposé

Catégorie	Instance	Coût obtenu	Valeur optimale	Gap (%)
Small	eil51	428.98	426	0.70
Small	berlin52	7544.37	7542	0.03
Small	st70	687.14	675	1.80
Medium	kroA100	21624.70	21282	1.61
Medium	ch150	6650.84	6528	1.88
Medium	d198	16087.63	15780	1.95
Large	lin318	45318.08	42029	7.83
Large	pr439	116522.78	107217	8.68
Large	pcb442	54753.86	50778	7.83
Large	rat575	7332.76	6773	8.27

Le tableau précédent présente les résultats obtenus avec la version hybride avancée de la recherche tabou. Afin d'évaluer l'apport de cette méthode, les résultats obtenus sont comparés avec ceux des configurations précédentes, notamment l'initialisation aléatoire (*cold start*), l'initialisation par la solution NN (*warm start*) et l'hybridation séquentielle.
Pour les petites instances (eil51, berlin52 et st70), la version hybride avancée ne permet pas d'obtenir d'amélioration significative par rapport aux configurations précédentes. Les solutions obtenues restent très proches de l'optimum, ce qui indique que la recherche tabou converge déjà rapidement pour des problèmes de petite taille.
Pour les instances de taille moyenne (kroA100, ch150 et d198), la recherche tabou hybride permet d'obtenir des solutions légèrement améliorées par rapport aux configurations précédentes. L'utilisation de stratégies d'intensification et de diversification contribue à explorer plus efficacement l'espace de recherche.
Pour les grandes instances (lin318, pr439, pcb442 et rat575), l'amélioration apportée par la version hybride est plus notable. Les mécanismes supplémentaires introduits dans l'algorithme permettent de mieux explorer les régions prometteuses de l'espace de recherche et de réduire progressivement l'écart avec les solutions optimales.
Globalement, ces résultats montrent que l'intégration de mécanismes hybrides dans la recherche tabou améliore la robustesse et la qualité des solutions obtenues, en particulier pour les instances de grande taille où l'espace de recherche est beaucoup plus vaste.

IV Analyse des résultats expérimentaux

1 Tableau global des résultats expérimentaux

Après l'exécution de l'algorithme sur les différentes instances du problème du voyageur de commerce (TSP), les résultats obtenus ont été regroupés dans un fichier `CSV`. Ce fichier contient pour chaque instance : la catégorie de taille, le nombre de villes, la valeur optimale connue, la solution obtenue par l'algorithme, le gap par rapport à l'optimal ainsi que le temps d'exécution.
Le tableau 16 présente l'ensemble des résultats obtenus.

Catégorie	Instance	Taille	Optimal	Solution	Gap (%)	Temps (s)
Small	eil51	51	426	428.982	0.700	0.819
Small	berlin52	52	7542	7544.366	0.031	0.512
Small	st70	70	675	687.142	1.799	0.266
Medium	kroA100	100	21282	21624.702	1.610	0.464
Medium	ch150	150	6528	6650.839	1.882	0.265
Medium	d198	198	15780	16087.629	1.949	0.130
Large	lin318	318	42029	45318.083	7.826	0.641
Large	pr439	439	107217	116522.778	8.679	0.348
Large	pcb442	442	50778	54753.861	7.830	0.593
Large	rat575	575	6773	7332.763	8.265	2.103

Table 16: Résultats expérimentaux obtenus sur différentes instances TSPLIB

L'analyse du tableau 16 montre que l'algorithme proposé obtient des solutions très proches de l'optimal pour les petites instances. Le gap reste inférieur à 2% pour les instances de petite et moyenne taille.
En revanche, pour les grandes instances, l'écart à l'optimal devient plus important, atteignant environ 8%. Ce comportement est attendu car l'augmentation du nombre de villes entraîne une croissance importante de l'espace de recherche, ce qui rend la résolution du problème plus difficile pour les méthodes heuristiques.
En termes de temps d'exécution, les résultats montrent que l'algorithme reste relativement rapide même pour les grandes instances, avec un temps maximal observé de 2.103 secondes pour l'instance `rat575`.

2 Analyse globale des performances

Afin d'évaluer les performances globales de l'algorithme, nous avons calculé le gap moyen ainsi que le temps moyen d'exécution sur l'ensemble des instances testées.

Gap moyen (%)	Temps moyen (s)
4.067	0.651

Table 17: Performances globales de l'algorithme

Les résultats montrent que le gap moyen obtenu est de 4.067%. Cela signifie que les solutions produites par l'algorithme restent relativement proches des solutions optimales connues.
Le temps moyen d'exécution est de 0.651 secondes, ce qui indique que l'algorithme permet d'obtenir des solutions de bonne qualité dans un temps de calcul relativement court.

3 Analyse par catégorie d'instances

Afin d'analyser l'influence de la taille des instances sur les performances de l'algorithme, les instances ont été regroupées en trois catégories : petites (*small*), moyennes (*medium*) et grandes (*large*).

Catégorie	Gap moyen (%)	Temps moyen (s)
Small	0.843	0.523
Medium	1.814	0.286
Large	8.150	0.923

Table 18: Performances moyennes par catégorie d'instances

On observe que le gap augmente progressivement avec la taille des instances. Pour les petites instances, les solutions obtenues sont très proches de l'optimal.

Pour les instances de taille moyenne, le gap reste relativement faible. En revanche, pour les grandes instances, l'écart à l'optimal devient plus important en raison de l'augmentation de la taille de l'espace de recherche. Le temps d'exécution reste toutefois raisonnable pour toutes les catégories d'instances.

4 Impact de la taille des instances sur les performances

Afin d'étudier l'influence de la taille des instances sur les performances de l'algorithme, nous avons analysé l'évolution du gap et du temps d'exécution en fonction du nombre de villes.

Taille	Gap (%)	Temps (s)
51	0.700	0.819
52	0.031	0.512
70	1.799	0.266
100	1.610	0.464
150	1.882	0.265
198	1.949	0.130
318	7.826	0.641
439	8.679	0.348
442	7.830	0.593
575	8.265	2.103

Table 19: Gap et temps d'exécution en fonction de la taille de l'instance

Les résultats montrent que l'écart à l'optimal augmente globalement avec la taille de l'instance. Ce comportement est attendu dans le problème du voyageur de commerce, car l'espace de recherche devient beaucoup plus grand lorsque le nombre de villes augmente.
En ce qui concerne le temps d'exécution, celui-ci reste relativement faible pour les petites et moyennes instances, mais peut augmenter pour certaines grandes instances.

5 Analyse graphique des performances

Afin de mieux visualiser l'impact de la taille des instances sur les performances de l'algorithme, une analyse graphique a été réalisée. Les figures suivantes présentent l'évolution du gap et du temps d'exécution en fonction du nombre de villes.

5.1 Évolution du gap en fonction de la taille de l'instance

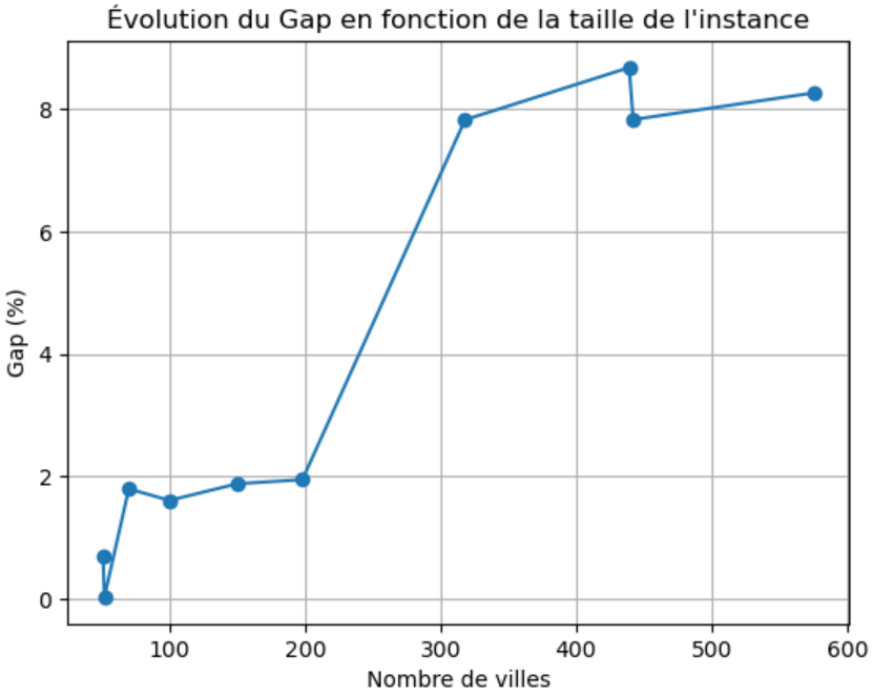


Figure 4: Évolution du gap en fonction du nombre de villes

La figure 4 illustre l'évolution du gap en fonction de la taille des instances du problème du voyageur de commerce.
On observe que pour les petites instances, le gap reste relativement faible, ce qui signifie que les solutions obtenues sont très proches de l'optimal. À mesure que la taille de l'instance augmente, le gap tend à croître de manière plus significative.
Ce comportement est attendu, car la complexité du problème augmente avec le nombre de villes. En effet, l'espace de recherche devient beaucoup plus vaste, ce qui rend la découverte de solutions proches de l'optimal plus difficile pour les méthodes heuristiques.

5.2 Évolution du temps d'exécution selon la taille des instances

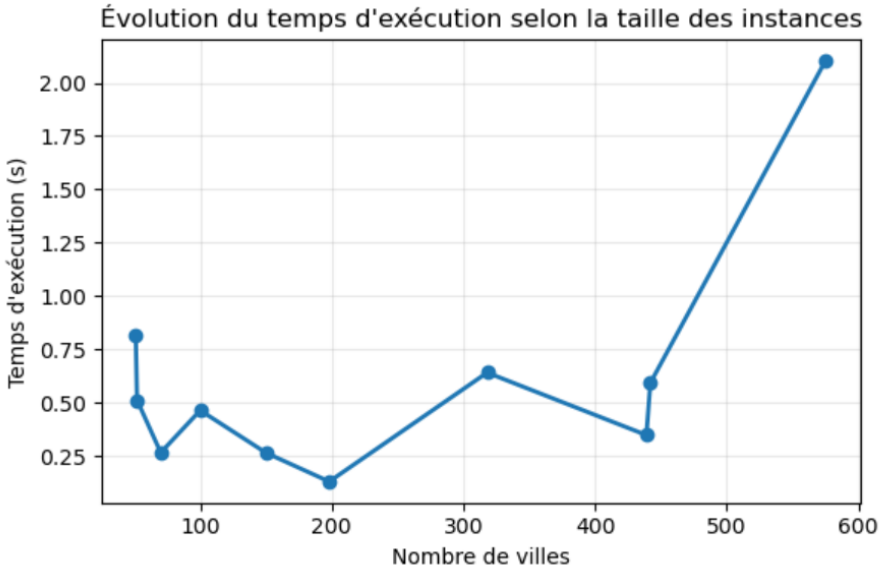


Figure 5: Évolution du temps d'exécution en fonction du nombre de villes

La figure 5 présente l'évolution du temps d'exécution de l'algorithme en fonction de la taille des instances. Les résultats montrent que le temps d'exécution n'augmente pas strictement de manière monotone avec la taille des instances. Ce comportement peut s'expliquer par la nature heuristique de l'algorithme utilisé ainsi que par les caractéristiques propres à chaque instance.
Cependant, une tendance générale à l'augmentation du temps d'exécution peut être observée pour les instances de grande taille. Cela est cohérent avec l'augmentation de la complexité du problème lorsque le nombre de villes devient plus important.

V Conclusion

Dans ce travail, nous avons étudié la résolution du problème du voyageur de commerce (TSP), l'un des problèmes classiques d'optimisation combinatoire, connu pour sa complexité de type NP-difficile. L'objectif était d'analyser l'efficacité des méthodes heuristiques et métaheuristiques pour obtenir des solutions de bonne qualité dans un temps de calcul raisonnable.

Dans un premier temps, l'heuristique du plus proche voisin a été implémentée afin de construire rapidement des solutions initiales. Les expériences ont montré que cette méthode permet de générer des tournées réalisables avec un temps de calcul très faible. Cependant, la qualité des solutions obtenues dépend fortement du choix de la ville de départ et reste limitée par la nature gloutonne de l'heuristique.

Afin d'améliorer ces solutions initiales, une métaheuristique de recherche tabou a été appliquée. Les différentes configurations testées, notamment l'initialisation aléatoire, l'initialisation par la solution NN (warm start) et l'hybridation séquentielle, ont permis d'améliorer significativement la qualité des solutions obtenues. Les résultats expérimentaux montrent que l'utilisation d'une solution initiale de bonne qualité permet d'accélérer la convergence de l'algorithme et de réduire l'écart par rapport aux solutions optimales.

L'analyse des performances a également mis en évidence l'influence de la taille des instances sur la qualité des solutions. Pour les petites et moyennes instances, les solutions obtenues sont très proches des valeurs optimales. En revanche, pour les grandes instances, l'écart à l'optimal devient plus important en raison de l'augmentation considérable de l'espace de recherche.

Globalement, les résultats obtenus confirment l'efficacité de l'approche hybride proposée, qui combine une heuristique de construction et une métaheuristique d'amélioration. Cette combinaison permet d'obtenir des solutions de bonne qualité tout en maintenant un temps d'exécution relativement faible.

Comme perspectives futures, il serait intéressant d'explorer d'autres stratégies d'hybridation, d'intégrer des méthodes d'apprentissage pour guider la recherche ou de comparer les performances obtenues avec d'autres métaheuristiques telles que les algorithmes génétiques ou le recuit simulé.

VI Annexes

Les expérimentations ont été réalisées à l'aide de notebooks Jupyter. Le code source correspondant aux différentes méthodes implémentées, notamment l'heuristique du plus proche voisin (NN) et la recherche tabou, est disponible dans les notebooks fournis au présent rapport.